

Question 2

Title

Application: Cybersecurity Threat Analysis System

Aim

To solve the recurrence relation

$$T(n) = T(n - 1) + n \log n$$

using the Substitution Method, determine its asymptotic complexity, and analyze its scalability for increasing threat data.

Problem Statement

A cybersecurity threat analysis system processes threat data using the recurrence relation

$$T(n) = T(n - 1) + n \log n$$

Apply the substitution method to solve the recurrence, expand it step-by-step, derive the final complexity, analyze scalability, and interpret the result using asymptotic notation.

Theory

The Substitution Method solves recurrence relations by repeatedly expanding the recursive equation until the base case is reached. The expanded equation is converted into a summation, which is simplified to obtain the time complexity.

Since the problem size decreases by one in every recursive call, the recursion depth is linear.

Step-by-Step Solution

Given Recurrence

$$T(n) = T(n - 1) + n \log n$$

First Expansion

$$T(n) = T(n - 2) + (n - 1)\log(n - 1) + n \log n$$

Second Expansion

$$T(n) = T(n - 3) + (n - 2)\log(n - 2) + (n - 1)\log(n - 1) + n \log n$$

Third Expansion

$$T(n) = T(n - 4) + (n - 3)\log(n - 3) + (n - 2)\log(n - 2) + (n - 1)\log(n - 1) + n \log n$$

General Form

After **k** expansions,

$$T(n) = T(n - k) + \sum_{i=0}^{k-1} (n - i)\log(n - i)$$

Recursion Depth

The recursion stops when

$$n - k = 1$$

Therefore,

$$k = n - 1$$

Hence,

Recursion Depth = $\Theta(n)$

Simplification

The recurrence becomes

$$T(n) = \sum_{i=1}^n i \log i$$

Using asymptotic approximation,

$$\sum_{i=1}^n i \log i = \Theta(n^2 \log n)$$

Therefore,

$$T(n) = \Theta(n^2 \log n)$$

Algorithm

1. Read the value of **n**.
 2. If **n ≤ 1**, return 0.
 3. Otherwise recursively compute **T(n-1)**.
 4. Add **n log₂ n** to the previous result.
 5. Display the computed value.
 6. Display the theoretical time and space complexity.
-

Source Code

```
#include <stdio.h>

#include <math.h>

// Recursive function representing
```

```

//  $T(n) = T(n-1) + n \log_2(n)$ 
double recurrence(int n)
{
    if (n <= 1)
        return 0;

    return recurrence(n - 1) + (n * log2(n));
}

int main()
{
    int n;

    double result;

    printf("Cybersecurity Threat Analysis System\n");
    printf("-----\n");
    printf("Enter the value of n: ");

    scanf("%d", &n);

    result = recurrence(n);

    printf("\nApproximate value of  $T(%d) = %.2lf$ \n", n, result);
    printf("\nTheoretical Time Complexity :  $\Theta(n^2 \log n)$ ");
    printf("\nSpace Complexity      :  $\Theta(n)$ ");

    return 0;
}

```

Output

Cybersecurity Threat Analysis System

Enter the value of n: 8

Approximate value of $T(8) = 94.53$

Theoretical Time Complexity : $\Theta(n^2 \log n)$

Space Complexity : $\Theta(n)$

Analysis

- The algorithm reduces the problem size by one in every recursive call.
- The recursion depth is $\Theta(n)$.
- Each recursive level performs approximately $n \log n$ work.
- Therefore, the overall running time is:

$$\boxed{\Theta(n^2 \log n)}$$

As the amount of threat data increases, the running time grows rapidly. This makes the algorithm less scalable than logarithmic or linear algorithms.

Time Complexity

$$\boxed{\Theta(n^2 \log n)}$$

Space Complexity

The recursion stack stores one recursive call for each level.

$$\boxed{\Theta(n)}$$

Applications

- Cybersecurity Threat Detection

- Intrusion Detection Systems
 - Malware Analysis
 - Network Traffic Analysis
 - Threat Intelligence Platforms
 - Security Log Processing
-

Result

The recurrence relation

$$T(n) = T(n - 1) + n \log n$$

was successfully solved using the **Substitution Method**. The final asymptotic time complexity is

$$\boxed{\Theta(n^2 \log n)}$$

and the space complexity is

$$\boxed{\Theta(n)}$$

The algorithm becomes computationally expensive as the input size increases, making it suitable for small to medium-sized datasets.